

캠퍼스런치

아키텍처·플로우 학습 가이드

작성일: 2026-08-20
목적: 코드 구조·데이터 흐름·기능별 호출 순서를 함수 이름과 용도로 이해하기
대상: develop 브랜치 (Apple/Google/Kakao 로그인, 혼잡도, 사장님, 리워드 포함)
관련 문서: docs/CODEBASE_DEBUG_GUIDE.pdf (디버깅·SQL·증상별 체크리스트)

목차

0. 꼭 알아야 하는 7가지 (핵심 개념)
1. 레이어 구조와 데이터 방향
2. 앱 시작 플로우 (main → init → stage)
3. 화면 전환 (stage) 체계
4. 로그인·회원가입 플로우 (이메일 / Google / Apple / Kakao)
5. 로그인 성공 후 공통 처리
6. 로그아웃
7. 흔잡도 제보 플로우
8. 사장님 플로우
9. Repository·Service 역할
10. 이름·DB 매핑 규칙
11. 새 기능 추가 순서

0. 꼭 알아야 하는 7가지

이 7가지만 먼저 몸에 익히면, 나머지 코드는 '어디에 붙는지'만 찾으면 됩니다.

★ ① Screen은 Supabase를 직접 호출하지 않는다

화면(LoginScreen, HomeScreen 등)은 AppProvider의 public 메서드만 호출합니다. DB·Auth·RPC는 data/ Repository 또는 services/ Service가 담당합니다. 이 규칙을 깨면 RLS·세션·에러 처리가 화면마다 달라져 버그가 납니다.

★ ② 전역 상태의 정보는 AppProvider 하나

로그인 여부, 닉네임, 식당 목록, stage(현재 화면), 북마크, 푸시 설정 등 앱 전체 상태는 AppProvider에만 있습니다. 화면 전용

UI state(텍스트 입력, 탭 인덱스 등)만 StatefulWidget 로컬에 둡니다.

★ ③ 화면 전환의 정보는 AppProvider.stage

main.dart의 _Root가 context.watch<AppProvider>().stage를 보고 Splash / Login / MainScreen 등을 switch합니다.

Navigator.push로 '앱 전체 단계'를 바꾸지 않습니다(인증번호 화면 같은 하위 라우트만 push).

★ ④ 로그인 성공의 정보는 Supabase 세션 + _onSupabaseSignedIn

이메일·Google·Apple·Kakao 모두 Supabase Auth 세션이 생긴 뒤 _onSupabaseSignedIn(user)로 합류합니다. 여기서 public.users 프로필 생성·닉네임·약관·stage·푸시·리워드를 한 번에 처리합니다.

★ ⑤ DB 프로필 테이블은 public.users (profiles 아님)

Supabase Auth의 auth.users와 1:1로 public.users가 연결됩니다. ProfileRepository.upsertFromAuthUser가 매핑 담당.

코드·SQL·문서 어디에도 profiles 테이블 이름을 쓰지 않습니다.

★ ⑥ 혼잡도 '정보'는 DB, 앱은 표시·폴백

제보 → crowd_reports INSERT → DB 트리거 recalculate → crowd_status 갱신이 정보입니다. 앱의

crowd_status_algorithm.dart는 병합·표시·오프라인 폴백용입니다. 앱만 고치고 SQL 트리거를 안 맞추면 화면과 DB가 어긋납니다.

★ ⑦ Provider public API는 String? 에러 반환 패턴

loginWithGoogle(), reportStatus(), register() 등은 성공 시 null, 실패 시 사용자에게 보여줄 한글 메시지 String을 반환합니다.

화면은 context.mounted 확인 후 SnackBar/텍스트로 표시합니다.

1. 레이어 구조와 데이터 방향

[Screens / Widgets]

LoginScreen, HomeScreen, MainScreen, ReportSheet ...

역할: UI 렌더링, 버튼 탭 → AppProvider 호출

|
v (이벤트 ↓)

[AppProvider] lib/providers/app_provider.dart

역할: 전역 state, stage, 비즈니스 규칙(콜다운·GPS), Repository 조합

|
v

[data/ Repository] + [services/ Service]

Repository: Supabase RPC·테이블 CRUD, row→Model 매핑

Service: 외부 SDK (Kakao, Google, Apple, FCM, 지도)

|
v

[Supabase] Auth + PostgreSQL + Realtime + Storage + Edge Functions

데이터 읽기: Supabase → Repository → AppProvider → Screen (watch)

사용자 액션: Screen → AppProvider → Repository/Service → Supabase

주요 파일 역할

- main.dart — 앱 진입, SDK 초기화, AppProvider 생성, stage별 루트 위젯
- providers/app_provider.dart — 앱 두뇌. stage, restaurants, 로그인, 제보, 사장님
- data/*_repository.dart — DB 접근·snake_case↔camelCase 매핑
- services/*_service.dart — Supabase 초기화, OAuth SDK, 푸시, 딥링크
- models/*.dart — Restaurant, Account, CrowdReport 등 불변 데이터 클래스
- config/env.dart — .env / native_keys에서 API 키 읽기

2. 앱 시작 플로우

앱이 켜질 때부터 첫 화면이 나올 때까지의 순서입니다.

1

→ main()
Flutter 진입점. dotenv·Firebase·Supabase 초기화

2

→ Env.loadReleaseConfig()
릴리즈 빌드용 키 fallback 로드

3

→ SupabaseService.initialize()
Supabase 클라이언트 생성. detectSessionInUri=false (딥링크는 AppLinkService)

4

→ AppProvider()..init()
세션 복구, 식당 목록 로드, stage 결정

5

→ AppLinkService.initialize(provider)
Universal Link / campuslunch:// 딥링크 수신

6

→ runDeferredStartupOnce
권한 동의 후 Kakao/Google/FCM/지도 SDK 지연 초기화

AppProvider.init() 상세

A

→ _loadRestaurantsFromSupabase()
식당·혼잡도 목록 백그라운드 fetch

B

→ SupabaseService.client.auth.currentSession
저장된 Supabase 세션 확인

C

→ _onSupabaseSignedIn(user)
세션 있으면 프로필·로컬 prefs 동기화

D

→ _restoreSessionFromPrefs()
SharedPreferences에서 닉네임·role 복원

E

→ _resolveStageAfterSplash(prefs)
다음 stage 결정 (권한/로그인/약관/앱)

F

→ _leaveSplash(nextStage)
최소 스플래시 시간 후 stage 변경 + notifyListeners

G

→ _bindAuthListener()
Supabase onAuthStateChanged 구독 (로그아웃·토큰 갱신)

★ triggerDeferredStartup() — OS 권한 팝업 타이밍

Kakao/Google SDK·FCM은 permissions_consent 화면에서 사용자가 '확인'을 누른 뒤 completePermissionsConsent() → triggerDeferredStartup()으로 초기화합니다. 앱 시작 직후 바로 초기화하면 OS 권한 팝업 순서가 꼬입니다.

3. 화면 전환 (stage) 체계

★ stage는 '앱 전체가 지금 어느 단계인가'를 나타내는 문자열

main.dart _RootState.build()가 context.watch<AppProvider>().stage를 읽고 switch(stage)로 루트 위젯을 바꿉니다. 로그인 성공·로그아웃·약관 완료 시 AppProvider 내부에서 _stage = '...' 후 notifyListeners()를 호출합니다.

stage 값 → 표시 화면
splash → SplashScreen
permissions_consent → PermissionsConsentScreen (최초 1회)
login → LoginScreen
legal_terms_consent → LegalTermsConsentScreen
usage_guide → UsageGuideScreen (최초 1회)
referral_code → ReferralCodeScreen (신규 가입)
app → MainScreen (하단 탭)
location_permission / notification_permission → (레거시 단계, 일부 기기)

stage 결정 함수

→ _resolveStageAfterSplash()
스플래시 후: 권한 미동의→permissions, 미로그인→login, 약관→legal, 아니면 app

→ _resolveStageForSession()
로그인 직후: 권한→약관→_postAppStage

→ _postAppStage()
usage_guide 미완→usage_guide, 추천인 대기→referral_code, 아니면 app

MainScreen 하단 탭

일반 유저: [홈] [지도] [MY]

사장님(hasOwnerTab): [사장님] [홈] [지도] [MY] — 인증 후 기본 탭=사장님

IndexedStack으로 탭 전환 (각 탭 state 유지)

4. 로그인·회원가입 플로우

모든 로그인은 '네이티브/SDK 인증 → Supabase 세션 → `_onSupabaseSignedIn`'으로 합류합니다.

4.1 이메일 로그인

UI

→ `LoginScreen._submit()`
이메일·비밀번호 검증 후 Provider 호출

→ `AppProvider.login(email, pw)`
Supabase `signInWithPassword`. `emailConfirmedAt` 없으면 실패

→ `SupabaseService.client.auth.signInWithPassword`
Supabase Auth 이메일 인증

→ `_onSupabaseSignedIn(user)`
프로필·세션·stage 공통 처리

4.2 이메일 회원가입 + OTP

UI

→ `LoginScreen._submit()` (회원가입 탭)
비밀번호 규칙·확인 검증

→ `AppProvider.register(email, pw, nick)`
가입 가능 여부 RPC 확인 후 OTP 메일 발송

→ `AuthRepository.checkEmailSignupStatus()`
RPC `email_signup_status`: `registered/pending/withdrawn`

→ `_sendSignupOtpEmail()`
Supabase가 6자리 인증번호 메일 발송

UI

→ `_EmailVerifyScreen._verify()`
사용자가 6자리 입력

→ `AppProvider.verifySignupOtp(email, code)`
OTP 검증 + 세션 생성

→ `SupabaseService.client.auth.verifyOTP`
`type=signup, token=6자리`

→ `_onSupabaseSignedIn(user, isNewSignup: true)`
신규 가입 후처리 (추천인·약관)

4.3 Google 로그인

UI

→ `LoginScreen._loginWithGoogle()`
로딩 상태, 에러 표시

→ `AppProvider.loginWithGoogle()`
에러 메시지 String? 반환

→ `GoogleAuthService.signInWithSupabase()`
Google SDK + Supabase `id_token` 교환

→ `generateRawNonce()` + `GoogleSignIn.authenticate()`
Google 계정 선택, `id_token`·`nonce` 발급

→ `AuthRepository.checkOAuthLoginEmail(email, 'google')`
RPC: 이메일 가입 계정과 충돌 차단

→ `auth.signInWithIdToken(provider: google, nonce)`
Supabase 세션 생성

→ `_onSupabaseSignedIn(user, isNewSignup: _isLikelyNewAccount(user))`
공통 후처리

4.4 Apple 로그인 (iOS만)

UI

→ `LoginScreen._loginWithApple()`
iOS에서만 버튼 표시 (`Platform.isIOS`)

→ `AppProvider.loginWithApple()`
AppleAuthService 호출 + 에러 처리

→ `AppleAuthService.signInWithSupabase()`
Sign in with Apple → Supabase

→ `generateRawNonce()` + `SignInWithApple.getIdCredential()`
Apple 시트, identityToken

→ `AuthRepository.checkOAuthLoginEmail(email, 'apple')`
이메일 충돌 확인

→ `auth.signInWithIdToken(provider: apple, nonce)`
Supabase 세션 (앱 .env에 Apple Client ID 불필요)

→ `_onSupabaseSignedIn(...)`
공통 후처리

4.5 Kakao 로그인 (코드 유지, UI 임시 숨김)

LoginScreen._showKakaoLogin = false 로 버튼만 숨김. Supabase issuer 허용 후 true로 바꾸면 즉시 복원.

UI

→ LoginScreen._loginWithKakao()
현재 UI에서 숨김 (_showKakaoLogin=false)

→ AppProvider.loginWithKakao()
KakaoAuthService + _onSupabaseSignedIn

→ KakaoAuthService.signInWithSupabase()
카카오 SDK → id_token → Supabase

→ _loginWithKakao()
카카오톡 또는 카카오계정 로그인, OAuthToken 반환

→ UserApi.instance.me()
카카오 프로필·이메일 (OpenID Connect 필요)

→ AuthRepository.checkOAuthLoginEmail(email, 'kakao')
이메일 충돌 확인

→ auth.signInWithIdToken(provider: kakao)
Supabase 세션 (현재 issuer 차단 이슈)

iOS

→ SceneDelegate + KakaoOAuthForwarder
kakao*://oauth URL을 Google/Flutter deeplink와 분리

→ AppLinkService.isKakaoOAuthCallback()
카카오 OAuth URL은 Supabase PKCE 처리에서 제외

★ OAuth 이메일 충돌 검사 (공통)

Google/Apple/Kakao 모두 로그인 전 AuthRepository.checkOAuthLoginEmail() RPC를 호출합니다. 같은 이메일로 이미 이메일 가입했거나 다른 provider로 가입한 경우 GmailBlocked 예외 → AppProvider가 한글 안내 메시지 반환.

5. 로그인 성공 후 공통 처리

★ `_onSupabaseSignedIn` — 모든 로그인의 종착역

이메일·Google·Apple·Kakao·세션 복구 모두 이 함수를 거칩니다. 여기를 이해하면 '로그인은 됐는데 닉네임/약관/탭이 이상하다' 버그를 빠르게 잡힐 수 있습니다.

- 1
→ `ProfileRepository.fetch(user.id)`
public.users 프로필 조회
- 2
→ `generateAvailableNickname()`
닉네임 없거나 placeholder면 자동 생성
- 3
→ `ProfileRepository.upsertFromAuthUser()`
public.users INSERT/UPDATE (auth.users와 동기화)
- 4
→ `_resolveOwnerRestaurantIds()`
사장님 매장 ID 목록 (restaurants.owner_id)
- 5
→ `_markReferralPromptPending()`
신규 가입이면 추천인 코드 화면 예약
- 6
→ `LegalConsentRepository.fetchHasRequiredConsents()`
약관 동의 DB 확인
- 7
→ `_saveSession()`
SharedPreferences 저장 + `_resolveStageForSession` → stage 결정
- 8
→ `recordAppSession()`
DAU/세션 analytics_events 기록
- 9
→ `_syncPushNotifications()`
푸시 설정 서버 동기화
- 10
→ `fetchMyReward()`
스탬프·기프티콘 상태 로드

`_saveSession()`가 하는 일

- `_isLoggedIn`, `_nickname`, `_accountId`, `_userRole` 메모리 갱신
- SharedPreferences에 로그인 플래그·닉네임·만료 시각 저장
- `_resolveStageForSession` → legal_terms / usage_guide / referral_code / app
- `notifyListeners()` → `_Root`·각 Screen rebuild

6. 로그아웃

- `AppProvider.logout()`
MyScreen 등에서 호출
- `FcmPushService.unregisterToken()`
FCM 토큰 서버에서 제거
- `KakaoAuthService.signOut()` / `GoogleSignIn.signOut()`
provider별 SDK 로그아웃
- `SupabaseService.client.auth.signOut()`
Supabase 세션 삭제
- `_clearSession(prefs)`
SharedPreferences 정리
- `_stage = 'login'`
로그인 화면으로 전환

7. 혼잡도 제보 플로우

★ 제보 정보 = DB 트리거

앱에서 status를 직접 restaurants에 쓰지 않습니다. submit_crowd_report RPC → crowd_reports INSERT → 트리거 recalculate
→ crowd_status 갱신.

표시 (읽기)

→ AppProvider._loadRestaurantsFromSupabase()
앱 시작·Realtime 시 목록 갱신

→ SupabaseRestaurantRepository.fetchAll()
restaurants + crowd_status + reports JOIN

→ _mergeRow() + computeStatusFromReports()
DB 값 + 클라이언트 알고리즘 병합

→ Restaurant.status
UI: 여유로움/약간혼잡/자리없음/영업안함

제보 (쓰기)

UI

→ ReportSheet → report_feedback.submitCrowdReportFeedback()
바텀시트에서 상태 선택

→ AppProvider.reportStatus(restaurantId, status)
쿨다운·GPS-source 판별

→ _ownsRestaurant() → source
본인 매장=owner, 그 외=user

→ _resolveVenueGps()
유저 제보: 매장 150m 이내 (디버그 빌드 우회)

→ SupabaseRestaurantRepository.reportStatusWithStamp()
RPC submit_crowd_report 호출

→ fetchMyReward() + repo.fetchAll()
스탬프 결과·목록 갱신

실시간

→ AppProvider._subscribeRealtime()
crowd_status / crowd_reports 변경 구독

→ repo.fetchAll()
변경 시 식당 목록 재로드

DB enum: crowd_level (normal/full/closed), crowd_source (user/owner/system)

Dart: CrowdLevelMapper가 enum ↔ 한글 UI 변환

8. 사장님 플로우

→ AppProvider.submitOwnerApplication(...)
사장님 인증 신청 (6자리 코드 방식 폐지, 서류 심사)

→ AppProvider.submitOwnerSeatUpdate(...)
입장 인원 제보 → RPC submit_owner_seat_update

→ OwnerScreen → reportStatus()
본인 매장만 owner source (5분 쿨다운 없음)

→ fetchOwnedRestaurantIds()
restaurants.owner_id 기준 소유 매장

hasOwnerTab=true 이면 mainScreen 첫 탭=사장님

9. Repository·Service 역할

AuthRepository	email_signup_status, oauth_login_email_check
ProfileRepository	public.users CRUD, 닉네임 중복
SupabaseRestaurantRepository	매장·혼잡도 fetch, submit_crowd_report
RewardRepository	스탬프·기프티콘, redeem_gifticon
CommunityRepository	게시글·댓글·차단
LegalConsentRepository	약관 동의 이력
AnalyticsRepository	DAU/MAU, 이벤트

SupabaseService	Supabase.initialize, client 싱글톤
KakaoAuthService	Kakao SDK + signInWithIdToken
GoogleAuthService	Google SDK + signInWithIdToken
AppleAuthService	Sign in with Apple + signInWithIdToken
AppLinkService	딥링크(초대, 이메일 콜백)
FcmPushService	FCM 토큰 등록·푸시 수신

★ Repository = DB/API 경계

Supabase row의 snake_case(image_url, crowd_level) → Model camelCase(imageUrl, crowdLevel) 변환은 Repository 한 곳에서만 합니다. Screen/Provider는 Model만 다룹니다.

10. 이름·DB 매핑 규칙

- Supabase 컬럼: snake_case (image_url, is_active, crowd_level)
- Dart Model 필드: camelCase (imageUrl, isActive, crowdLevel)
- 구역: Restaurant.area / DB area (region 사용 금지)
- 프로필 테이블: public.users (profiles 금지)
- 위치 권한 상태: AppProvider.locationMode
- 지도 마커: RestaurantKakaoMap.showMyLocationMarker
- rename 시 grep으로 전체 검색 후 정의+call site 한 번에 수정

11. 새 기능 추가 순서 (PDF 가이드 8.1)

1. supabase/*.sql 테이블·RPC·RLS·트리거
2. lib/models/ Dart 모델
3. lib/data/ Repository (매핑·RPC 호출)
4. lib/providers/ AppProvider public 메서드
5. lib/screens/ UI

아키텍처·DB 매핑을 바꾸면 tool/generate_codebase_guide_pdf.py 도 갱신

★ 학습 순서 추천

① main.dart + AppProvider.stage → ② LoginScreen + _onSupabaseSignIn → ③ reportStatus + submit_crowd_report → ④ MainScreen 탭 구조. 이 순서로 코드를 읽으면 전체 그림이 빠르게 잡힙니다.

관련 문서

- docs/CODEBASE_DEBUG_GUIDE.pdf — SQL 순서, 증상별 디버깅, APK/OAuth
- docs/KAKAO_SUPABASE_SETUP.md — 카카오·issuer 이슈
- docs/APPLE_SUPABASE_SETUP.md — Apple Provider 설정
- docs/GOOGLE_SUPABASE_SETUP.md — Google OAuth 설정